The American Statistician

ISSN: 0003-1305 (Print) 1537-2731 (Online) Journal homepage: www.tandfonline.com/journals/utas20

A Survey on Large Language Model-based Agents for Statistics and Data Science

Maojun Sun, Ruijian Han, Binyan Jiang, Houduo Qi, Defeng Sun, Yancheng Yuan & Jian Huang

To cite this article: Maojun Sun, Ruijian Han, Binyan Jiang, Houduo Qi, Defeng Sun, Yancheng Yuan & Jian Huang (16 Oct 2025): A Survey on Large Language Model-based Agents for Statistics and Data Science, The American Statistician, DOI: 10.1080/00031305.2025.2561140

To link to this article: https://doi.org/10.1080/00031305.2025.2561140

THE AMERICAN STATISTICIAN
2025, VOL. 00, NO. 0, 1–14: Statistical Computing and Graphics
https://doi.org/10.1080/00031305.2025.2561140

A Survey on Large Language Model-based Agents for Statistics and Data Science

Maojun Sun[a], Ruijian Han[a], Binyan Jiang[a], Houduo Qi[a,b], Defeng Sun[a], Yancheng Yuan[b], and Jian Huang[a,b]

[a]Department of Data Science and Artificial Intelligence, The Hong Kong Polytechnic University, Hung Hom, Kowloon, Hong Kong; [b]Department of Applied Mathematics, The Hong Kong Polytechnic University, Hung Hom, Kowloon, Hong Kong

ABSTRACT

In recent years, data science agents powered by Large Language Models (LLMs), known as "data agents," have shown significant potential to transform the traditional data analysis paradigm. This survey provides an overview of the evolution, capabilities, and applications of LLM-based data agents, highlighting their role in simplifying complex data tasks and lowering the entry barrier for users without related expertise. We explore current trends in the design of LLM-based frameworks, detailing essential features such as planning, reasoning, reflection, multi-agent collaboration, user interface, knowledge integration, and system design, which enable agents to address data-centric problems with minimal human intervention. Furthermore, we analyze several case studies to demonstrate the practical applications of various data agents in real-world scenarios. Finally, we identify key challenges and propose future research directions to advance the development of data agents into intelligent statistical analysis software.

ARTICLE HISTORY
Received December 2024
Accepted August 2025

KEYWORDS
Data agents; Data analysis; Generative AI; Natural language interaction; Statistical software

1. Introduction

As nearly every aspect of society becomes digitized, data analysis has emerged as an indispensable tool across various industries (Inala et al. 2024). For instance, financial institutions leverage data analysis to make informed decisions about stock trends (Institute 2011; Provost and Fawcett 2013), hospitals use it to monitor patients' health conditions (Waller and Fawcett 2016), and companies employ it to develop strategic plans (Chen, Chiang, and Storey 2012). Despite its widespread utility, data analysis is often perceived as a challenging field with a significant "entry barrier" (Jordan and Mitchell 2015; Cao 2017), typically requiring knowledge in areas such as statistics, data science, and computer science (Kitchin 2014). Since the release of SPSS (IBM 1968) in 1968, followed by SAS (Inc. 1976), Matlab (MathWorks 1984), Excel (Microsoft 1985), Python (Foundation 1991), R (for Statistical Computing 1995), PowerBI (Microsoft 2013), and other specialized data analysis tools and programming languages, these advancements have significantly aided professionals in conducting statistical experiments and data analysis. Moreover, they have made data analysis more accessible to a broader range of practitioners (Witten, Frank, and Hall 2016).

The general data analysis process typically involves several key steps. Initially, data is collected from studies or extracted from databases and imported into tools such as Excel. Next, software like Excel or programming languages such as Python and R are employed to clean and analyze the data, aiming to extract valuable insights. Subsequently, data visualization is performed to make these insights more accessible and understandable. For more complex tasks, such as statistical inference and predictive analysis, statistical and machine learning models are often necessary. This involves data processing, feature engineering, modeling, evaluation, and more. Upon completing the analysis, a final report is usually drafted to summarize the findings and insights. However, for individuals without expertise in statistics, data science, and programming, data analysis remains a high-barrier task.

The barriers to data analysis primarily exist in the following areas:

- Lack of systematic statistical training: Individuals without a background in statistics may find it challenging to understand which types of analysis are feasible, even when data is presented to them. As data and models become increasingly complex, gaining a solid understanding of current statistical techniques typically requires at least a Master's level of statistical training.
- Software limitation: Simple data analysis tools like Excel are inadequate for complex scenarios, such as predictive analysis or analyzing data from enterprise databases. Conversely, advanced programming languages for data analysis, such as Python and R, require prior programming knowledge, which can be a barrier for many users.
- Challenges in domain-specific problems: In specialized fields like protein or genetic data analysis, general data scientists may find it difficult to perform effective analysis due to a lack of domain-specific knowledge.

CONTACT Yancheng Yuan *yancheng.yuan@polyu.edu.hk* Department of Applied Mathematics, The Hong Kong Polytechnic University, Hung Hom, Kowloon, Hong Kong; Jian Huang *j.huang@polyu.edu.hk* Department of Applied Mathematics, The Hong Kong Polytechnic University, Hung Hom, Kowloon, Hong Kong.
Supplementary materials for this article are available online. Please go to *www.tandfonline.com/r/TAS*.

- Difficulty in integrating domain knowledge: Corresponding to the last point, domain experts often lack the data science and programming skills needed to quickly incorporate their expertise into data analysis tools. For example, PSAAM (Steffensen, Dufault-Thompson, and Zhang 2016) is software designed for the curation and analysis of metabolic models, yet a biologist researching metabolism might find it challenging to integrate this analytical method into common data analysis tools like Excel or R.

With the rise of generative AI, new opportunities have emerged in statistics and data science. LLM-based data agents are gradually addressing existing challenges while introducing a new paradigm for approaching data analysis tasks.

An "AI agent" (or LLM agent) refers to an autonomous or semi-autonomous software system powered by AI models such as LLMs. These agents can interpret natural language instructions, plan and execute tasks, and interact with users or other systems to complete complex workflows (Cheng et al. 2024).

Specifically, we define an LLM-based data agent as an autonomous or semi-autonomous software system powered by LLMs, capable of understanding natural language instructions, planning and executing data-centric tasks, and interacting with users or external tools to accomplish complex objectives-from exploratory data analysis to machine learning model development. In this article, the terms "LLM-based data science agent," "LLM-based data agent," and "data science agent" are collectively referred to as "data agent" for simplicity.

This survey explores recent advancements in data agents and highlights data analysis performed by various agents through a series of case studies. In Section 2, we briefly discuss the opportunities introduced by recent developments in generative AI. Section 3 reviews and categorizes recent work on data science agents. We then present several case studies in Section 4. Section 5 examines the challenges and future directions in this field, followed by our discussion in Section 6. Finally, we present our conclusions in Section 7.

2. Opportunities Brought by Generative AI

The rise and potential of generative AI, particularly Large Language Models (LLMs) or vision language models (VLMs) in the field of data science and analysis have gained increasing recognition in recent years. In addition to understand text, LLMs are also trained to understand tabular data, allowing them to effectively extract insights, identify patterns, and draw meaningful conclusions from tables (Dong and Wang 2024). Consequently, LLMs have emerged as powerful tools capable of significantly enhancing and transforming a variety of data-driven applications and workflows (Nejjar et al. 2023; Tu et al. 2023; Cheng, Li, and Bing 2023). Recent research has focused on designing LLM-based data science agents (data agents) to automatically address data science tasks through natural language, as demonstrated by tools like ChatGPT-Advanced Data Analysis (ChatGPT-ADA) (OpenAI 2023), LAMBDA (Sun et al. 2024) and Colab Data Science Agent (Google 2025).

The emergence of data agents offers a potential solution to the previously mentioned challenges, as they lower the entry barrier for users who lack programming or statistical knowledge. By providing an intuitive interface that harnesses the capabilities of LLMs, users can request analyses using natural language, and the data agents can interpret these instructions, access relevant data, and autonomously apply appropriate analytical techniques. For example, a user might request, "Calculate the sales growth in different regions from 2021 to 2028, generate a bar chart to visualize the results, and provide key insights." With this simplified instruction, data agents can automatically extract, analyze, visualize, and report data, reducing the requirement for technical expertise and fostering a more efficient workflow. This significantly lowers the entry barriers for individuals unfamiliar with traditional data analysis tools and methods.

Furthermore, by embedding specialized knowledge into LLMs, data agents can potentially overcome challenges faced by data scientists in fields like genomics, where domain expertise is crucial (Cao 2017). Simultaneously, domain experts who may lack data science or programming skills can rely on data agents to seamlessly integrate their expertise into data analysis workflows. This ability to bridge the gap between domain expertise and data science has the potential to advance interdisciplinary research and decision-making in complex scenarios (Figure 1).

3. LLM-based Data Science Agents

3.1. Overview

LLM-based data agents leverage the powerful natural language understanding and generation capabilities of LLMs to autonomously tackle complex data analysis tasks. Figure 3 illustrates a commonly used framework for these agents.

In this framework, the LLM serves as the core of the entire system, driving its performance and reliability. As such, the capabilities of the LLM are critical to the system's effectiveness, with advanced models like GPT-4 often being used. Data analysis typically involves multiple steps, especially when addressing complex tasks. Techniques such as Planning, Reasoning, and Reflection help ensure that the LLM processes these tasks with greater logical coherence and makes optimal use of its knowledge.

In the architecture, the LLM generates the code for a given data analysis task, executes it, and retrieves the corresponding results. This requires an execution environment, represented by the Sandbox, which safely isolates the code execution process. The Sandbox allows users to run programs and access files without risking the underlying system or platform. It includes pre-installed programming environments and software, such as Python, R, Jupyter, and SQL Server.

A user-friendly interface is also essential to improving usability. An intuitive interface not only attracts users but also enables them to quickly engage with and use the system effectively.

3.2. Evolution of Data Science Agent

Research on data agents began gaining momentum in 2023. Chandel et al. (2022) trained and evaluated a model within a Jupyter Notebook to predict code based on given commands and results. Soon after, it was discovered that LLMs, such as GPT, could generate accurate code for basic data analysis. With

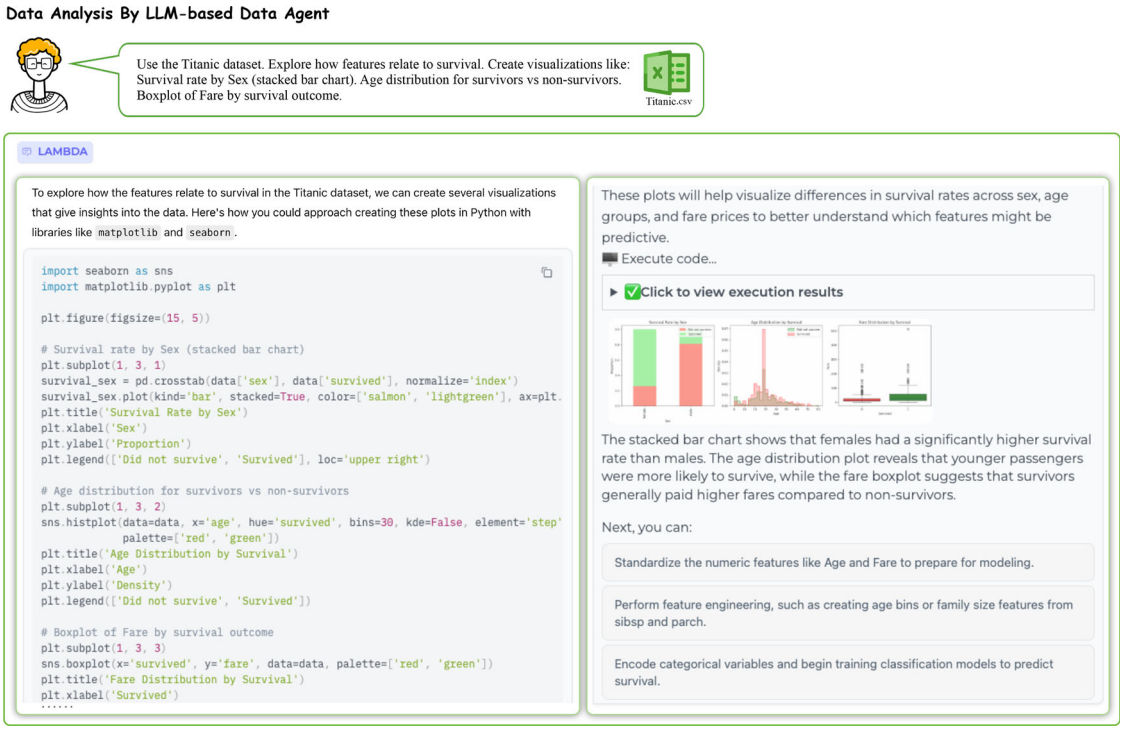

Figure 1. New paradigm of data analysis brought by generative AI.

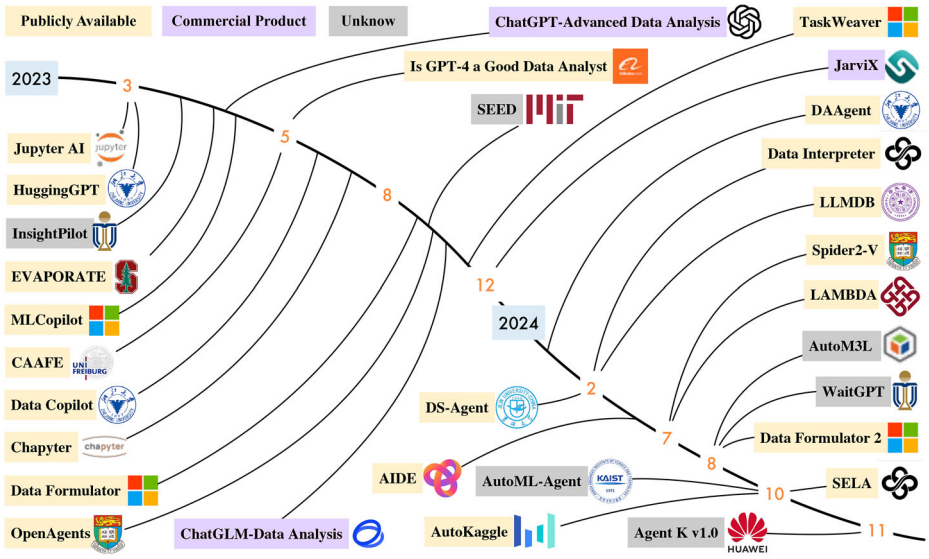

Figure 2. Timeline of selected related works from 2023.

the rise of the LLM-based agent, researchers began designing special data agents for automating data science and analysis tasks by human language. Figure 2 shows some selected works from 2023, while Table 1 illustrates some key characteristics.

3.3. User Interface

The user interface is crucial for attracting users at first glance. Current research on user interface design can be broadly categorized into four types: Integrated Development Environment-based (IDE-based), Independent System, Command line-based (Command-based), and Operation System-based (OS-based).

IDE-based. Integrated Development Environments (IDEs) such as Jupyter provide convenient tools for data science and analysis. Recent efforts, including Colab Data Science Agent (Google 2025), Jupyter-AI (jupyterlab 2023), Chapyter (chapyter 2023), and MLCopilot (Zhang et al. 2023a), have incorporated LLMs into Jupyter environments. For example, Colab Data Science Agent enables planning, automatic code cell generation, execution, and result presentation in the notebook. This approach is particularly popular because it allows users to review, edit, and run code directly.

Independent System. Some works have focused on developing independent systems equipped with user interfaces. For example, ChatGPT introduced a streamlined, intuitive conversational system-a model of interaction that has been widely adopted in subsequent projects. In the context of data analysis tasks, beyond

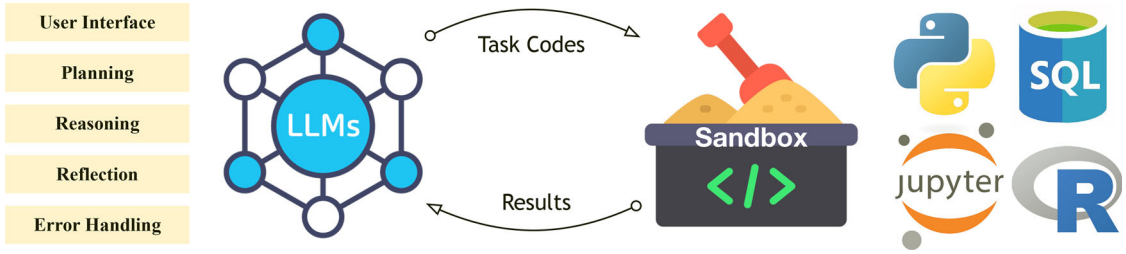

Figure 3. An architecture of an LLM-based data agent. The diagram illustrates the interaction between LLMs and a sandbox environment. On the left, key components of LLMs are highlighted, including User Interface, Planning, Reasoning, Reflection, and Error Handling. The sandbox, positioned centrally, serves as a controlled environment for executing task codes and generating results. On the right, various tools and software that can be pre-installed in the sandbox, such as Python, SQL, Jupyter, and R, indicate the diverse ecosystems where LLM-powered agents can operate.

Table 1. Characteristics of selected data agents.

Data agents	Methods	User interface	Planning	Human in the loop	Self-correcting	Expandable
ChatGPT-ADA (OpenAI 2023)	Conversational	System	Linear	✗	✔	✗
Data Copilot (Zhang et al. 2023b)	End-to-end	System	Linear	✗	✔	✗
Jupyter AI (jupyterlab 2023)	Conversational	IDE-based	Basic IO	✔	✗	✗
MLCopilot (Zhang et al. 2023a)	Conversational	IDE-based	Basic IO	✔	✗	✗
Chapyter (chapyter 2023)	Conversational	IDE-based	Basic IO	✔	✗	✗
Openagents (Xie et al. 2023)	Conversational	System	Linear	✗	✗	✔
JarviX (Chen et al. 2024)	End-to-end	–	–	–	–	–
DS-Agent (Guo et al. 2024)	End-to-end	CLI	Linear	✗	✔	–
Spider2-V (Cao et al. 2024)	End-to-end	OS-Based	–	✗	✔	–
ChatGLM-DA (GLM 2024)	Conversational	System	Linear	✗	✔	✗
TaskWeaver (Qiao et al. 2023)	End-to-end	CLI & System	Linear	✗	✔	✔
Data Interpreter (Hong et al. 2024)	End-to-end	CLI	Hierarchical	✔	✔	✔
LAMBDA (Sun et al. 2024)	Conversational	System	Basic IO	✔	✔	✔
Data Formulator 2 (Wang et al. 2024a)	Conversational	System	Basic IO	✗	✔	–
AutoM3L (Luo et al. 2024)	End-to-end	–	–	✗	–	✔
SELA (Chi et al. 2024)	End-to-end	CLI	Hierarchical	✗	✔	–
AIDE (Jiang et al. 2024)	End-to-end	CLI	Hierarchical	✗	✔	–
AutoKagle (Li et al. 2024)	End-to-end	CLI	Linear	✔	✔	✔
AutoML-Agent (Trirat, Jeong, and Hwang 2024)	End-to-end	–	Linear	–	✔	–
Agent K v1.0 (Grosnit et al. 2024)	End-to-end	–	Linear	–	✔	✗
GPT-4o (OpenAI 2024)	End-to-end	System	–	✗	✔	✔
AutoGen Studio (Wu et al. 2023)	End-to-end	System	Linear	✗	✔	✔
Colab Data Science Agent (Google 2025)	End-to-end	IDE-based	Linear	✔	✔	✗

NOTE: Methods can be categorized into Conversational and End-to-End approaches. Conversational methods support interactive dialogue with iterative user feedback, whereas End-to-End approaches rely on a single prompt, with the agent autonomously planning and solving the problem. The user interface can be categorized into IDE-based, Systems, CLI, and OS-based. The term "Human-in-the-Loop" indicates that humans can intervene in the data agent's workflow, such as modifying code in situations where automatic processes are inadequate. "Self-Correcting" refers to the agent's ability to automatically identify and correct errors within the workflow through reflection. Finally, "Expandable" denotes the data agent's capacity to incorporate customized tools or knowledge. "–" indicates that the attribute is either not mentioned in the article or could not be observed from the provided resources.

basic text-based input and output, several systems have introduced specialized features, such as visualization, report generation, and file download options, to simplify user interactions. For instance, LAMBDA (Sun et al. 2024) facilitates easy data review by enabling intuitive data display after users upload their data. Data Formulator 2 (Wang et al. 2024a) further enhances the iterative process of creating data visualizations through a multimodal interface, combining graphical user interface (GUI) elements with natural language inputs, allowing users to specify their visualization intentions with both precision and flexibility. WaitGPT (Xie et al. 2024) addresses the challenge of understanding and verifying LLM-generated code by transforming raw code into an interactive, step-by-step visual representation. This allows users to comprehend, validate, and adjust specific data operations, actively guiding and refining the analysis process.

Command Line-based. Works like Data Interpreter (Hong et al. 2024) and TaskWeaver (Qiao et al. 2023) using command-line interfaces (CLI) in their works. For researchers and experienced users, it provides greater flexibility and control over the system, allowing users to execute a wide range of functions in the command line and customize their actions. Besides, command-based interfaces often require less computational overhead compared to graphical user interfaces, making them more efficient.

OS-based. OS-based agents, such as UFO (Zhang et al. 2024), are designed to operate directly within an operating system environment, allowing them to control a wide range of system tasks and resources. Similarly, Spider2-V (Cao et al. 2024) simulates the typical workflow of a data scientist by mimicking actions such as clicking, typing, and writing code, providing an OS-level interactive experience that closely resembles how humans manage data science tasks. However, while OS-based agents like Spider2-V lay a solid foundation for user interaction, achieving full automation of the data science workflow remains an ongoing challenge (Cao et al. 2024).

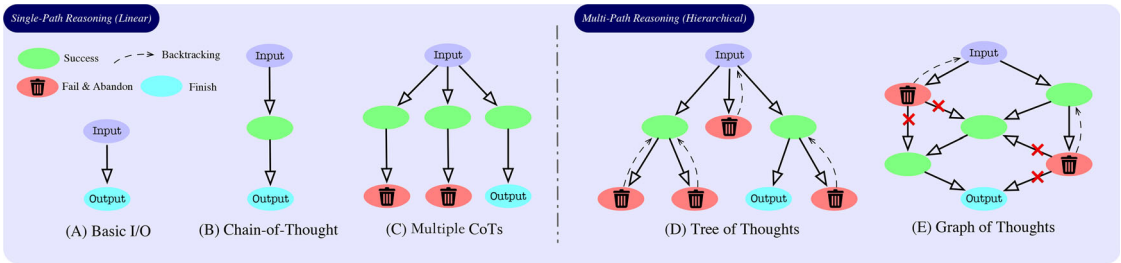

Figure 4. Commonly used planning and reasoning strategies in LLM-based data agents for organizing tasks or solving problems. Each node represents a sub-task in the roadmap.

3.4. Planning, Reasoning, and Reflection

Planning, Reasoning, and Reflection often play crucial roles in guiding the actions of data agents. In particular, planning and reasoning emphasize the generation of a logically structured sequence or roadmap of actions and thought processes to systematically address problems step by step (Huang et al. 2024b; Hong et al. 2024). Complex tasks often require a step-by-step approach to ensure effective resolution, while simpler tasks can be handled without such detailed breakdowns. Recently, GPT-4o (OpenAI 2024) introduces a planning architecture that integrates external tools and decomposes complex tasks into structured sub-tasks, enabling more accurate and controllable multi-step reasoning.

Some approaches focus on building conversational data agents (Zhang et al. 2023a, 2023b; Sun et al. 2024), where users interact with the agent over multiple rounds to complete a task. In these cases, under human supervision, complex planning is not necessary, as guidance can simplify decision-making and adjust the workflow dynamically. Some of these works operate in a Basic I/O mode. On the other hand, End-to-end data agents (Qiao et al. 2023; Guo et al. 2024; Hong et al. 2024; Chi et al. 2024; Jiang et al. 2024; Li et al. 2024; Trirat, Jeong, and Hwang 2024; Grosnit et al. 2024) are designed to allow users to issue a single prompt that encompasses all requirements. In these cases, the agent employs planning, reasoning, and reflection to iteratively complete all tasks autonomously.

Recent research in planning has introduced two main approaches: Linear Structure Planning (or Single Path Planning/Reasoning) and Hierarchical Structure Planning (or Multiple Path Planning/Reasoning). Figure 4 illustrates some recent planning methodologies like Chain-of-Thought (CoT) (Wei et al. 2022), ReAct (Yao et al. 2022), Tree-of-Thoughts (ToT) (Yao et al. 2024), and Graph-of-Thoughts (GoT) (Besta et al. 2024).

Linear Structure Planning. In linear structure planning, a task is decomposed into a sequential, step-by-step process. For example, DS-Agent (Guo et al. 2024) uses Case-Based Reasoning to retrieve and adapt relevant insights from a knowledge base of past successful Kaggle solutions. This approach allows the agent to learn from previous experiences and continuously improve its performance. Similarly, AutoML-Agent (Trirat, Jeong, and Hwang 2024) adopts a retrieval-augmented planning (RAP) strategy to generate diverse plans for AutoML tasks. By leveraging the knowledge embedded in LLMs, information retrieved from external APIs, and user requirements, RAP allows the agent to explore a wider range of potential solutions, leading to more optimal plans.

Hierarchical Structure Planning. Simple linear planning is often insufficient for complex tasks. Such tasks may require hierarchical and dynamic, adaptable plans that can account for unexpected issues or errors in execution (Hong et al. 2024). For instance, Hong et al. (2024) uses a hierarchical graph modeling approach that breaks down intricate data science problems into manageable sub-problems, represented as nodes in a graph, with their dependencies as edges. This structured representation enables dynamic task management and allows for real-time adjustments to evolving data and requirements. Additionally, they further introduce "Programmable Node Generation," to automate the generation, refinement, and verification of nodes within the graph, ensuring accurate and robust code generation. AIDE (Jiang et al. 2024) employs Solution Space Tree Search to iteratively improve solutions through generation, evaluation, and selection components. Similarly, SELA (Chi et al. 2024) combines LLMs with Monte Carlo Tree Search (MCTS) to enhance AutoML performance. It starts by using LLMs to generate insights for various machine learning stages, creating a search space for solutions. MCTS then explores this space by iteratively selecting, simulating, and back-propagating feedback, enabling the discovery of optimal pipelines. Agent K v1.0 (Grosnit et al. 2024) employs a structured reasoning framework with memory modules, operating through multiple phases. The first phase, automation, handles data preparation and task setup, generating actions through structured reasoning. The second phase, optimization, involves solving tasks and enhancing performance using techniques such as Late-Fusion Model Generation and Bayesian optimization. The final phase, generalization, uses a memory-driven system for adaptive task selection.

Reflection. Reflection enables an agent to evaluate past actions and decisions, adjust strategies, and improve future task performance. This process is essential for self-correction and debugging during task execution. For example, Wang et al. (2024b) employs trajectory filtering to train agents that can learn from interactions and enhance their self-debugging capabilities. This technique involves selecting trajectories in which the model initially makes errors but successfully corrects them through self-reflection in subsequent interactions. Similarly, Data-copilot (Zhang et al. 2023b) and LAMBDA (Sun et al. 2024) use self-reflection based on code execution feedback to address errors. If a compilation error occurs, the agents

repeatedly attempt to revise the code until it runs successfully or a maximum retry limit is reached. This iterative process helps ensure code correctness and usability.

3.5. Multi-Agent Collaboration

Multi-agent System (MAS) enable task decomposition through role assignment. In this setup, agents communicate, negotiate, and share information to optimize their collective performance (Xi et al. 2023). It offers several advantages over single-agent setups. First, they reduce redundant and complex context accumulation by isolating responsibilities across agents. Second, each agent instance can be powered by a different language model, opening opportunities to specialize models for domain-specific expertise. For example, in LAMBDA (Sun et al. 2024), a dedicated Programmer Agent is responsible for code generation, while noisy error outputs are handled separately by an Inspector Agent. This separation helps the Programmer Agent avoid context overload, simplifies historical trace management, and ultimately improves response accuracy.

AutoGen introduces a programming framework specifically designed for constructing MAS (Wu et al. 2023). Furthermore, AutoML-Agent (Trirat, Jeong, and Hwang 2024) involves the Agent Manager, Prompt Agent, Operation Agent, Data Agent, and Model Agent-that together cover the entire pipeline, from data retrieval to model deployment. OpenAgents (Xie et al. 2023) consisted of agents such as the Data Agent, Plugins Agent, and Web Agent. Similarly, AutoKaggle (Li et al. 2024) employs agents like Reader, Planner, Developer, Reviewer, and Summarizer to manage each phase of the process, ensuring comprehensive analysis, effective planning, coding, quality assurance, and detailed reporting. These collaborating mode help decentralized the complicated task, allowing each agent to focus on its specific role, thereby enhancing the overall efficiency and effectiveness of the data analysis process.

3.6. Knowledge Integration

Integrating domain-specific knowledge into data agents presents a challenge (Dash et al. 2022; Sun et al. 2024). For example, when a domain expert has specialized knowledge, such as specific protein analysis code, the agent system are expected able to incorporate and apply this knowledge effectively. One approach is tool-based, where the expert's analysis code is treated as a tool that is recognizable by the LLM (Xie et al. 2023). When the agent encounters a relevant problem, it can call upon the appropriate tool from its library to execute the specialized analysis. Another method involves the Retrieval-Augmented Generation (RAG) technique (Lewis et al. 2020), where relevant code is first retrieved and then embedded within the context to facilitate in-context learning. LLM-based agents can also access and interact with external knowledge sources, such as databases or knowledge graphs, to augment their reasoning capabilities (Wang et al. 2024b).

Sun et al. (2024) proposes a Knowledge Integration method that builds on this concept. In LAMBDA, analysis codes are parsed into two parts: descriptions and executable code. These are then stored in a knowledge base. When the agent receives a task, it retrieves the relevant knowledge based on the similarity between the task description and the descriptions stored in the knowledge base. The corresponding code is then used for in-context learning (ICL) or back-end execution, depending on the configuration. This approach enables agents to effectively leverage domain-specific knowledge in relevant scenarios.

3.7. Benchmarks for Evaluating Data Agents

Evaluating the performance of data agents is crucial for understanding their effectiveness and reliability. Current benchmarks primarily rely on deterministic output comparisons, where an LLM processes a task, generates code, and is evaluated based on the final execution results. For example, DS-1000 (Lai et al. 2023) provides a large-scale benchmark of 1000 realistic problems spanning seven core Python data science libraries, with execution-based multi-criteria evaluation and mechanisms to reduce memorization bias. MLAgentBench (Huang et al. 2024a) introduces a benchmark focused on machine learning research workflows by constructing an LLM-agent pipeline. Furthermore, InfiAgent-DABench (Hu et al. 2024) presents a end-to-end benchmark for evaluating the capabilities of data agents, the tasks require agents to end-to-end solving complex tasks by interacting with an execution environment. However, for tasks such as data visualization, the outputs are often difficult to compare directly. Designing effective evaluation strategies for data visualizations remains an open and important question.

3.8. System Design and Other Related Works

Recent advancements in interactive data science systems highlight a variety of approaches in system design, with LLMs and structured frameworks significantly enhancing the user experience across key areas such as data visualization, task specification, predictive modeling, and data exploration. Notable systems like VIDS (Hassan, Knipper, and Santu 2023), Data-Copilot (Zhang et al. 2023b), InsightPilot (Ma et al. 2023), and JarviX (Liu et al. 2023) exemplify diverse design principles tailored to these specific functions. For instance, Data-Copilot adopts a code-centric approach, generating intermediate code to process data and subsequently transforming it into visual outputs, such as charts, tables, and summaries (Zhang et al. 2023b).

Other frameworks emphasize workflow automation. InsightPilot integrates an "insight engine" that guides data exploration, reducing LLM hallucinations and enhancing the accuracy of exploratory tasks (Ma et al. 2023). JarviX, in combination with MLCopilot (Zhang et al. 2023a), contributes to automated machine learning by merging LLM-driven insights with AutoML pipelines. Additionally, in the domain of database management, systems like LLMDB (Zhou, Zhao, and Li 2024) improve efficiency and reduce hallucinations and computational costs during tasks such as query rewriting, database diagnosis, and data analytics. In terms of data visualization, MatPlotAgent (Yang et al. 2024) transforms raw data into clear, informative visualizations by leveraging both code-based and multi-modal LLMs.

Moreover, Data Formulator 2 (Wang et al. 2024a) organizes user interactions into "data threads" to provide context and facilitate the exploration and revision of prior steps. A similar approach is seen in WaitGPT (Xie et al. 2024), which transforms

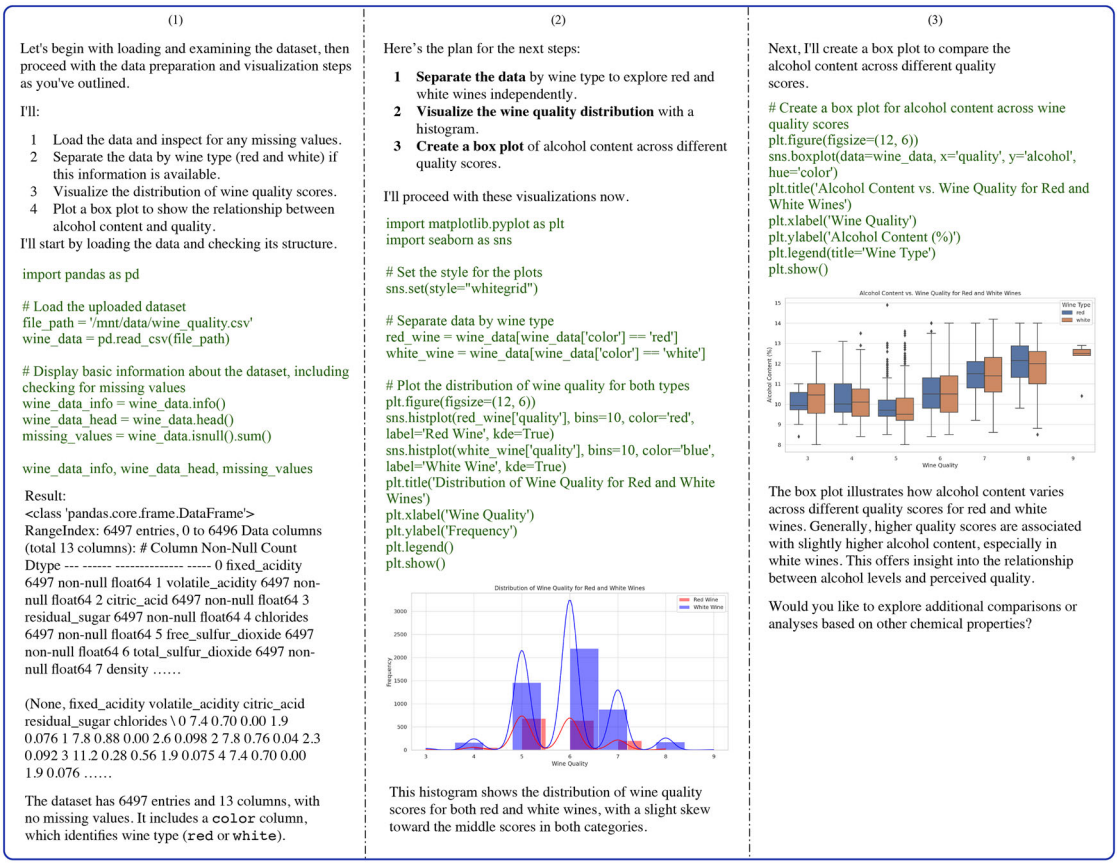

Figure 5. Partial dialogue from the ChatGPT-Advanced Data Analysis in Case Study 1. Items 1–4 list the work done by ChatGPT in each step.

raw code into an interactive visual representation. This provides a step-by-step visualization of LLM-generated code in real-time, allowing users to understand, verify, and modify individual data operations. SEED (Chen et al. 2024) combines LLMs with methods like code generation and small models to produce domain-specific data curation solutions. HuggingGPT (Shen et al. 2024), on the other hand, uses LLMs to coordinate a variety of expert models from platforms such as Hugging Face, solving a broader range of AI tasks across multiple modalities.

Lastly, in terms of industry applications, lots of companies have used agents in the business analysis. For example FUTU use AI to analyze the stock market and provide investment advice (FUTU 2024). Julius (Julius 2025) facilitates data science education by building a bridge that allowing professors to create interactive workflows for lessons, which can be shared with students for a seamless teaching experience through natural language interaction.

4. Data Analysis Through Natural Language Interaction: Case Studies

In this section, we present a series of case studies conducted by a diverse range of agents, each illustrating the new data analysis paradigm facilitated through natural language interaction. These case studies demonstrate how this approach enables users to engage with data more intuitively and effectively, breaking down traditional barriers to data accessibility and understanding. By leveraging natural language processing, these agents can interpret and respond to complex queries, providing insights that are both comprehensive and easily digestible. Through these examples, we aim to highlight the transformative potential of natural language interaction in data analysis.

4.1. Case Study 1: Exploratory Data Analysis and Model Building by Conversational Data Agents

In this case study, we used ChatGPT and LAMBDA to demonstrate exploratory data analysis (EDA) and a simple model building process. Specifically, we first used ChatGPT to explore the effect of alcohol content on the quality of different types of wine, focusing on both red and white varieties. Then, we used LAMBDA to illustrate an interactive modeling process and automatically generate analysis reports.

We used the Wine Quality dataset, a tabular dataset with dimension 4898×11. The goal is to examine how 10 covariates in this dataset affect the wine quality rating. We employed ChatGPT-ADA to conduct EDA and visualize the influence of alcohol content on wine quality ratings. Figure 5 illustrates the detailed planning and problem-solving process.

GPT-ADA first analyzed the problems and then outlined a step-by-step plan to solve the tasks. The entire workflow proceeded smoothly, with the code running efficiently to load the data, check for missing values, and generate visualizations, with each step delivering accurate results. Its ability to interpret data and provide insights significantly streamlined the analytical process. Finally, it provided insights into the relationship between quality scores and alcohol content.

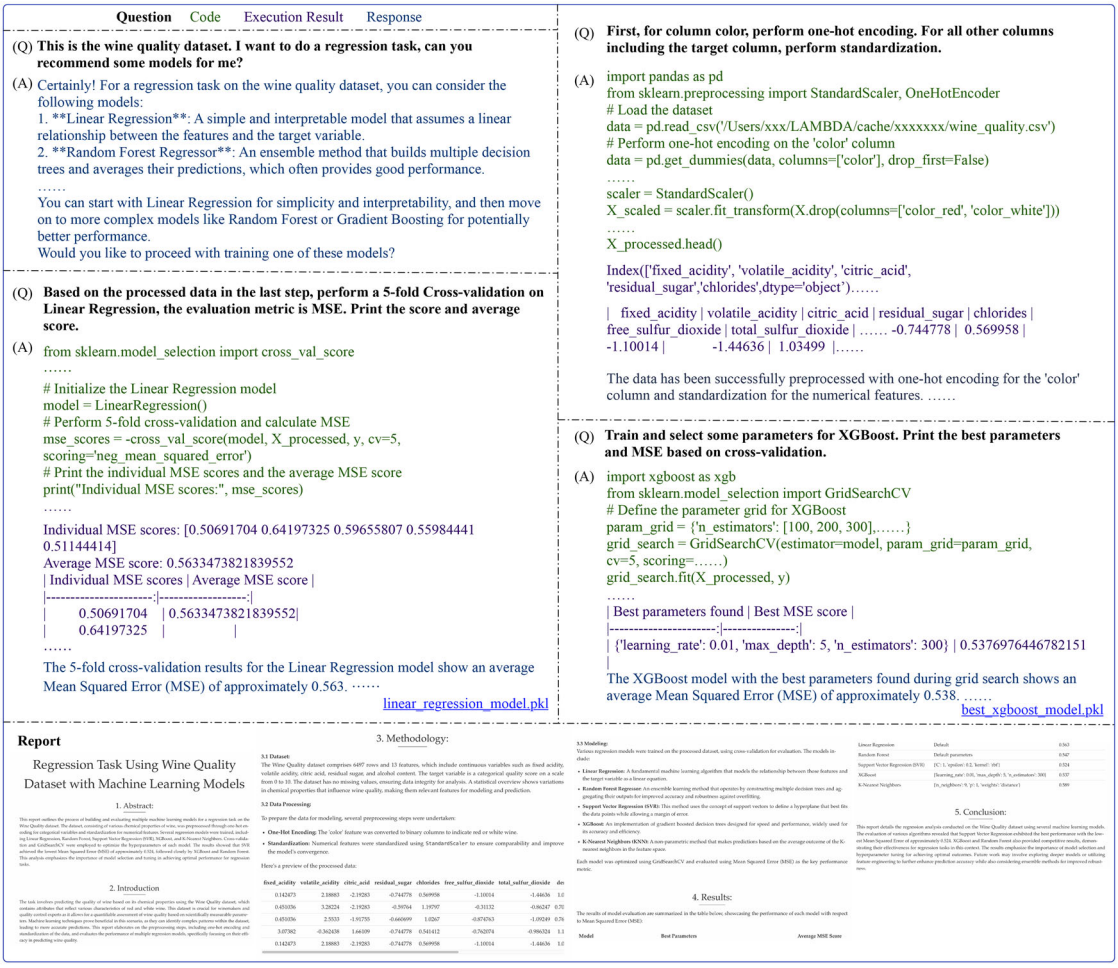

Figure 6. Conversational machine learning and report generation by LAMBDA. Excerpt from a partial dialogue.

Next, we train a set of models to predict wine quality using LAMBDA. LAMBDA facilitates an interactive analysis process, enabling us to perform tasks such as data processing, feature engineering, model training, parameter tuning, and evaluation through a series of guided conversations. Finally, we used LAMBDA's built-in report generation feature to compile a analysis report, which includes details of the tasks completed in the conversation history. The analysis process, including the conversation and the generated report, is presented in Figure 6.

As beginner-level users, we first asked LAMBDA to recommend some models, and it suggested advanced options like XGBoost. Next, we tasked LAMBDA with basic data preprocessing, which it handled correctly. We then trained and evaluated the recommended models using 5-fold cross-validation, a task LAMBDA performed exceptionally well, even providing download links for the resulting models. Finally, we used LAMBDA's report generation feature to create a structured and comprehensive report that effectively captured the key insights.

This example demonstrates the effectiveness of conversational data agents like ChatGPT and LAMBDA in streamlining the data visualization and machine learning workflow, particularly for users without programming experience.

4.2. Case Study 2: Residual Diagnostics and Heteroscedasticity Testing

To examine the ability of LLM-based data agents to perform statistically rigorous regression diagnostics, we prompted LAMBDA and GPT-4o to conduct a linear regression analysis using the Auto MPG dataset, a tabular data with dimension of 398 *times* 7. The goal was to predict `mpg` (miles per gallon) based on vehicle characteristics, notably `horsepower` and `weight`. The prompt and response of LAMBDA are detailed in the Figure 7.

LAMBDA correctly loaded the dataset, performed appropriate preprocessing (e.g., handling non-numeric entries), and fit a linear model using `statsmodels`. It then computed and visualized residuals, followed by executing the Breusch-Pagan test for heteroscedasticity. The test output included the LM statistic and associated p-value, indicating a strong violation of the homoscedasticity assumption.

The residual plot visually confirmed increasing residual variance with larger fitted values. LAMBDA also summarized next steps, suggesting robust standard errors or model transformation to address heteroscedasticity. This example demonstrates LAMBDA's ability to execute, interpret, and communicate statistically meaningful diagnostics in a flexible code-first environment. Besides, GPT-4o was also able to complete the same task successfully; further details

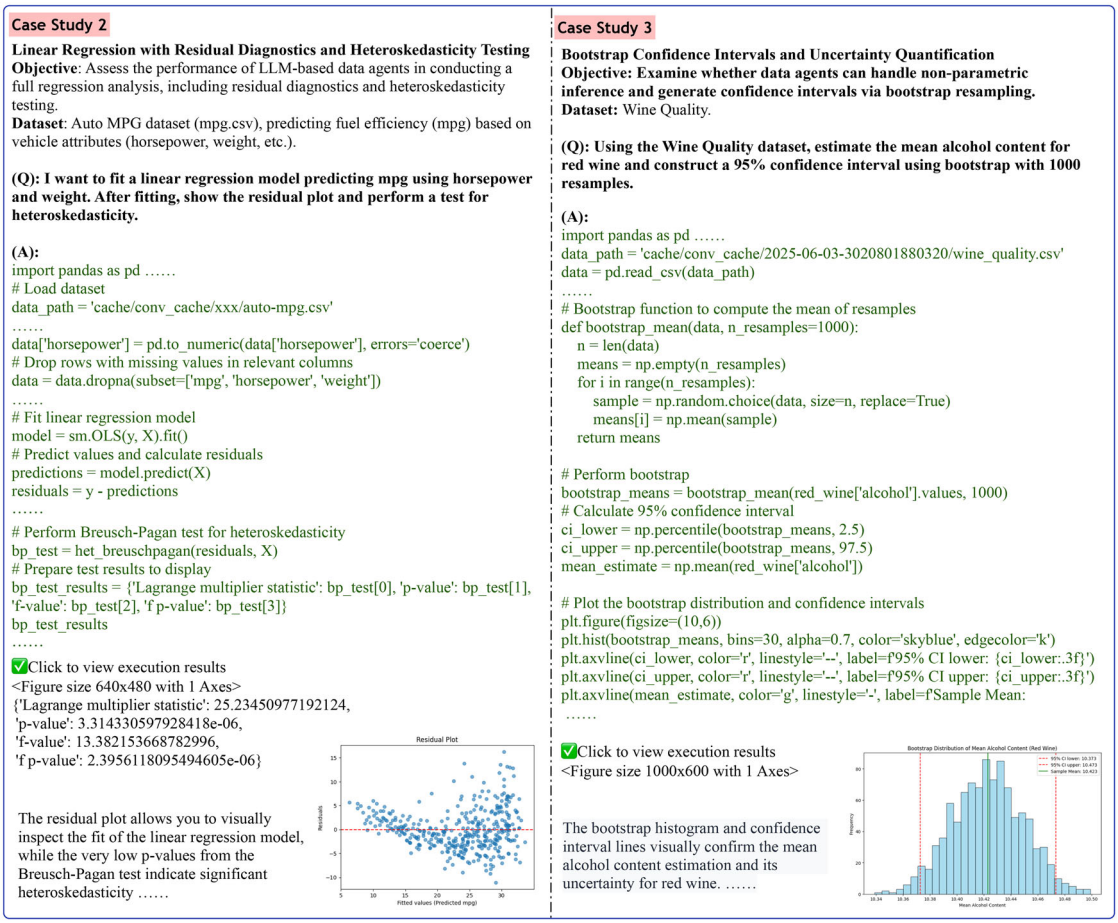

Figure 7. Partial dialogue from residual diagnostics and heteroscedasticity testing, and bootstrap confidence interval estimation.

and chat transcripts can be found in the supplementary materials.

4.3. Case Study 3: Bootstrap Confidence Intervals

In this case study, we assessed whether LLM-based data agents can perform nonparametric inference through bootstrap resampling. Using the Wine Quality dataset, the task was to estimate the average alcohol content for red wine and construct a 95% confidence interval using 1000 bootstrap resamples. Figure 7 shows the interaction with LAMBDA for completing this task.

LAMBDA successfully filtered the dataset to isolate red wines, extracted the `alcohol` variable, and implemented the bootstrap routine by repeatedly sampling with replacement. It then computed the empirical 2.5th and 97.5th percentiles of the bootstrapped means to form the confidence interval. The agent also produced a histogram showing the bootstrap distribution, overlaid with the CI bounds and sample mean.

This case illustrates that LAMBDA is capable of performing robust uncertainty quantification and generating high-quality visual explanations without relying on strict parametric assumptions. GPT-4o also successfully completed this task; its outputs and detailed interactions are included in the supplementary materials.

We found that different prompting may lead to differences in implementation details, such as the choice of hyperparameters or types of plots.

4.4. Case Study 4: Expandability of Data Agents

In many situations, we encounter tasks that cannot be handled effectively using LLMs because their training data do not include the necessary knowledge for such tasks. In these cases, if a data agent is designed to be extensible, manual tool expansion or knowledge integration can address this limitation. In this case study, we demonstrate how both the Data Interpreter and LAMBDA leverage integration mechanisms to incorporate additional packages or domain-specific knowledge.

Tools Integration in Data Interpreter. In this example, our objective is to extract submission deadlines for AI conferences from a public website[1] and save the results. We prompted the agent with the target URL and the desired output format. The agent successfully identified relevant information such as conference names and deadlines and generated structured output. The complete workflow, including prompt, execution, and results, is shown in Figure 8.

[1] *https://aideadlin.es*

Figure 8. Creating and using the customized tool in the Data Interpreter. Excerpt from a partial dialogue.

In this example, the Data Interpreter began with an initial plan. For each sub-task, it recommended relevant tools with a score indicating their suitability. The system then decided whether to use the suggested tool. For instance, it used `scrape_web_playwright` for a web-scraping task. This iterative recommendation and tool selection process continued until all sub-tasks were completed, addressing limitations in LLMs' built-in abilities and knowledge.

Knowledge Integration in LAMBDA. In this example, we consider the problem of training a Fixed Point Non-Negative Neural Network (FPNNN), which is defined as a neural network that maps nonnegative vectors to nonnegative vectors. We train a FPNNN with MNIST data. First, we integrated the code into the knowledge base. Then, we defined the model as `Core` and delineated the `Core` function, which directly accepts parameters, and the `Runnable` function, which was defined and executed separately. Figure 6 presents the configuration, prompt, and problem-solving process.

LAMBDA first retrieved the relevant code from the knowledge base, and then its `Core` function was presented in the context. By modifying the core code, LAMBDA generated the correct code and completed the task successfully (Figure 9).

5. Challenges and Future Directions

In this section, we highlight some challenges and suggest future directions in using LLMs or LLM-based data agents for statistical analysis.

5.1. Challenges in the Capabilities of LLMs

LLMs function as the "brain" of a data agent, interpreting user intent and generating structured plans to carry out data analysis tasks. For a data agent to be effective, it must possess advanced knowledge in statistics, data science, and programming, enabling it to support users throughout the analytical process.

Advanced Models. Current state-of-the-art models like GPT-4 show strong performance on undergraduate-level mathematics and statistics problems, yet struggle with more advanced, graduate-level tasks (Frieder et al. 2023). Additionally, the success rate of fully automating complete data workflows with current agents remains low (Cao et al. 2024). This suggests that enhancements in LLMs, particularly in knowledge of statistics and data analysis, are still needed.

Multi-Modality and Reasoning. A key challenge for current LLMs lies in processing multi-modal inputs, including charts, tables, and code, which are essential to data analysis workflows (Inala et al. 2024). Future advancements may improve the ability to perform reasoning across mixed modalities, such as generating visualizations by replicating the style of an input visualization.

5.2. Challenges in Statistical Analysis

Intelligent Statistical Analysis Software. While established tools such as SPSS and R are highly mature, data agents have the potential to transform statistical analysis through intelligent

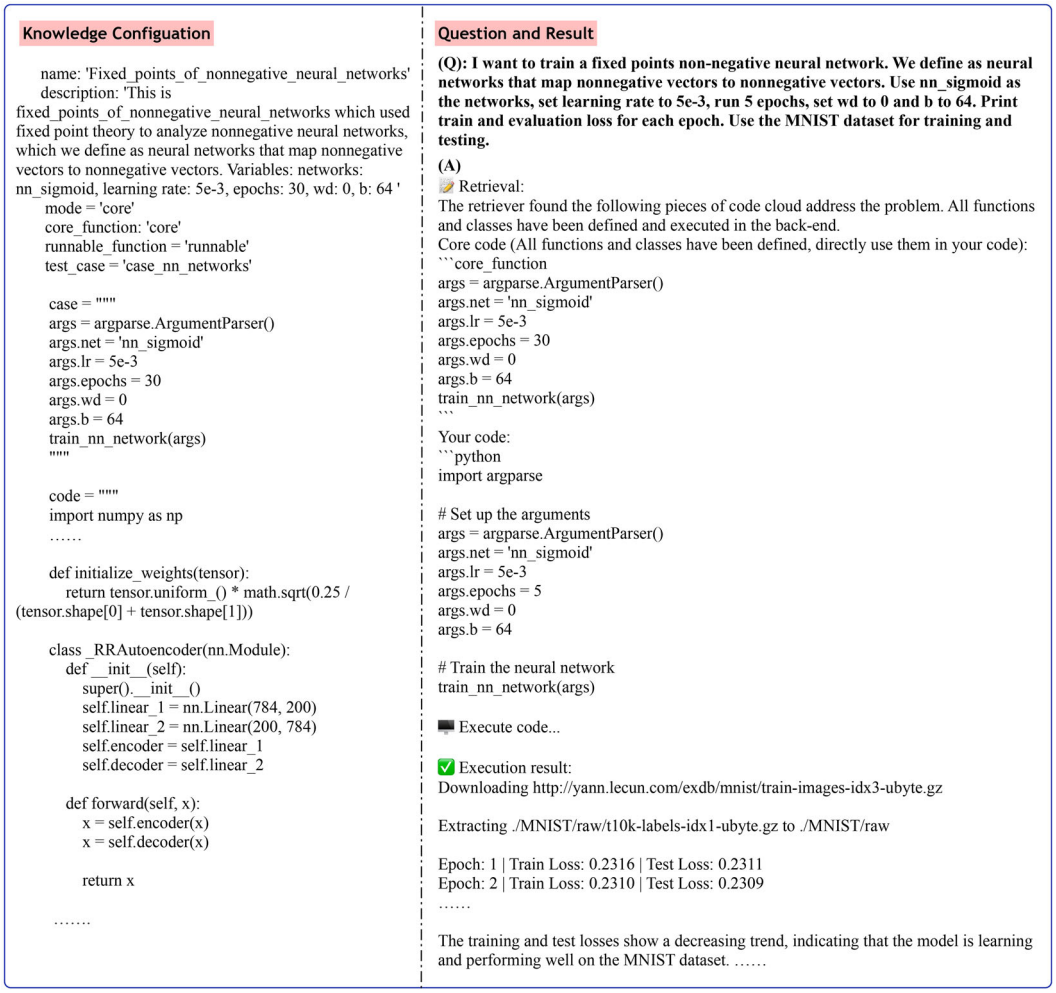

Figure 9. Integrating knowledge of FPNNNs in LAMBDA. Excerpt from a partial dialogue.

assistance. To realize this vision, agents must support flexible package integration, facilitate contributions from domain experts, and remain aligned with evolving programming ecosystems. Such a collaborative framework could accelerate innovation in the field. Furthermore, by guiding users and recommending appropriate methods, data agents can enhance research efficiency and expand access to advanced statistical techniques.

Incorporating Other Large Models into Statistical Analysis. Statistical analysis of complex data is increasingly leveraging representations generated by large models for research purposes. For example, in predicting the tertiary structure of proteins, LLMs can use representations of primary and secondary structures-capabilities that traditional statistical software such as Matlab and R currently lack. Similarly, in the analysis of electronic health records, LLMs are being used to construct meaningful representations that facilitate downstream analysis. If data agents can effectively harness domain-specific knowledge models, they have the potential to significantly advance statistical and data science research, enabling more sophisticated analyses and fostering deeper insights across scientific disciplines.

5.3. Challenges in Real-World Adoption

Although the data agents have shown great potential in improving the accessibility of data analysis, there are still several challenges that need to be addressed for real-world adoption.

Tradeoff Between Hardware and Privacy. First, deploying large language models often requires high-performance computing resources. Running these models on CPU-only machines results in slow inference. API-based solutions also raise concerns about data privacy and security, as sensitive information may be transmitted to external servers. This is especially critical in fields such as healthcare and finance, where data confidentiality is paramount. Therefore, developing lightweight, expert-level data science models that can run efficiently on local machines without compromising performance is essential.

High-concurrency System. High-concurrency environments pose significant scalability issues. In client-server architectures where each user session is associated with an isolated sandbox for secure code execution, the server may experience substantial resource strain under heavy load. Maintaining a large number of concurrent sandboxes can overwhelm system resources, leading to degraded performance or system instability. Therefore, the

design of efficient scheduling algorithms to manage limited computational resources across multiple sandbox instances becomes critical.

Integration with Existing Workflows. While data agents excel in lowering the barrier to entry for non-programmers, they currently lack the flexibility and debugging capabilities of traditional IDEs. This makes them less suitable for complex, customized workflows that require iterative development and fine-grained control. A promising direction is to support the seamless export of an agent's actions (Sun et al. 2024), such as executed code, into IDEs like Jupyter Notebooks, which can serve as a bridge for smoother integration with conventional tools and workflows.

6. Discussion

6.1. Model Level Reproducibility

While data agents are generally robust to variations in prompt phrasing and can reliably complete the intended analytical tasks, we observed notable differences in their reasoning processes and implementation details. For example, when prompted to perform regression diagnostics, different phrasings such as "analyze residuals" versus "check model assumptions" resulted in the same core analysis but with different statistical tests or plotting choices. Similarly, in visualization tasks, one prompt might produce a bar chart while another yields a pie chart, depending on how the goal is described. Even for model training, default hyperparameters, such as learning rate or number of iterations, could vary slightly across prompts, leading to differences in performance metrics. These variations do not typically prevent task completion but can impact result interpretability, especially in rigorous statistical workflows where consistency across runs is critical.

6.2. System Level Reproducibility

Experiment Setting. Experiment reproducibility can be enhanced through careful experiment designs. For example, LAMBDA (Sun et al. 2024) incorporates built-in mechanisms to export the full execution history into executable formats such as Jupyter Notebooks. When combined with proper experiment controls, such as setting random seeds, these exports enable end-to-end reproducibility of experimental results. In addition, designing human-in-the-loop mechanisms allows users to inspect, edit, or revise the code generated by LLMs during the problem-solving process. This interactive approach further supports reproducibility by enabling manual correction and verification of intermediate steps.

Version Control and Workflow Management. Version control tools such as Git can enhance reproducibility by tracking changes in code, data, and prompts, making it easier to reproduce results and collaborate with others. Furthermore, workflow management systems like Snakemake and Nextflow allow users to define and automate each step of the analysis pipeline, ensuring that processes can be reliably repeated. When used alongside data agents, these tools can greatly improve both reproducibility and transparency. However, most current data agents lack native support for these tools, presenting opportunities for future development.

7. Conclusion

This survey has explored the recent progress of LLM-based data science agents. These agents have shown great potential in making data analysis more accessible to a wider range of users, even those with limited technical skills. By leveraging the capabilities of LLMs, they are able to handle various data analysis tasks, from data visualization to machine learning, through natural language interaction.

However, as discussed, they also face several challenges. In terms of model capabilities, improvements are needed in domain-specific knowledge and multi-modal handling. For intelligent statistical analysis software, seamless package management and community building are crucial. Additionally, effectively integrating other large models into statistical analysis and addressing data infrastructure and evaluation issues remain important areas for future development.

Overall, while LLM-based data science agents have made significant strides, continuous research and innovation are required to overcome the existing challenges and fully realize their potential in revolutionizing the field of data analysis.

Supplementary Materials

The supplementary materials provide details of the datasets used in the main paper and additional case studies described therein.

Acknowledgments

The authors are grateful to the Editor, Associate Editor and two anonymous reviewers for their valuable comments and suggestions, which significantly improved the quality of the article.

Disclosure Statement

The authors report there are no competing interests to declare.

Funding

This work was funded by the Centre for the Mathematical Foundations of Generative AI and the research grants from The Hong Kong Polytechnic University (P0046811). The research of Ruijian Han was partially supported by The Hong Kong RGC grant (14301821) and The Hong Kong Polytechnic University (P0044617, P0045351, P0050935). The research of Binyan Jiang was partially supported by The Hong Kong RGC grant (15302722). The research of Houduo Qi was partially supported by the Hong Kong RGC grant (15309223) and The Hong Kong Polytechnic University (P0045347). The research of Defeng Sun and Yancheng Yuan was partially supported by the Research Center for Intelligent Operations Research at The Hong Kong Polytechnic University (P0051214). The research of Jian Huang was partially supported by The Hong Kong Polytechnic University (P0042888, P0045417, P0045931).

ORCID

Ruijian Han http://orcid.org/0000-0002-9225-2218

References

Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Podstawski, M., Gianinazzi, L., Gajda, J., Lehmann, T., Niewiadomski, H., Nyczyk, P., et al. (2024), "Graph of Thoughts: Solving Elaborate Problems with Large Language Models," in *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 38), pp. 17682–17690. [5]

Cao, L. (2017), "Data Science: Challenges and Directions," *Communications of the ACM*, 60, 59–68. [1,2]

Cao, R., Lei, F., Wu, H., Chen, J., Fu, Y., Gao, H., Xiong, X., Zhang, H., Mao, Y., Hu, W., Xie, T., Xu, H., Zhang, D., Wang, S., Sun, R., Yin, P., Xiong, C., Ni, A., Liu, Q., Zhong, V., Chen, L., Yu, K., and Yu, T. (2024), "Spider2-v: How Far Are Multimodal Agents from Automating Data Science and Engineering Workflows?" [4,10]

Chandel, S., Clement, C. B., Serrato, G., and Sundaresan, N. (2022), "Training and Evaluating a Jupyter Notebook Data Science Assistant," arXiv preprint arXiv:2201.12901. [2]

chapyter. (2023), "Chapyter." available at *https://github.com/chapyter/chapyter*. [3,4]

Chen, H., Chiang, R. H., and Storey, V. C. (2012), "Business Intelligence and Analytics: From Big Data to Big Impact," *MIS Quarterly*, 36, 1165–1188. [1]

Chen, Z., Cao, L., Madden, S., Kraska, T., Shang, Z., Fan, J., Tang, N., Gu, Z., Liu, C., and Cafarella, M. (2024), "Seed: Domain-Specific Data Curation with Large Language Models," arxiv 2023. arXiv preprint arXiv:2310.00749. [4,7]

Cheng, L., Li, X., and Bing, L. (2023), "Is gpt-4 a Good Data Analyst?" arXiv preprint arXiv:2305.15038. [2]

Cheng, Y., Zhang, C., Zhang, Z., Meng, X., Hong, S., Li, W., Wang, Z., Wang, Z., Yin, F., Zhao, J., et al. (2024), "Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects," arXiv preprint arXiv:2401.03428. [2]

Chi, Y., Lin, Y., Hong, S., Pan, D., Fei, Y., Mei, G., Liu, B., Pang, T., Kwok, J., Zhang, C., et al. (2024), "Sela: Tree-Search Enhanced LLM Agents for Automated Machine Learning," arXiv preprint arXiv:2410.17238. [4,5]

Dash, T., Chitlangia, S., Ahuja, A., and Srinivasan, A. (2022), "A Review of Some Techniques for Inclusion of Domain-Knowledge into Deep Neural Networks," *Scientific Reports*, 12, 1040. [6]

Dong, H., and Wang, Z. (2024), "Large Language Models for Tabular Data: Progresses and Future Directions," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '24, pp. 2997–3000, New York, NY, USA. Association for Computing Machinery. [2]

for Statistical Computing, R. F. (1995), "R: A Language and Environment for Statistical Computing." [1]

Foundation, P. S. (1991), "Python Programming Language." [1]

Frieder, S., Pinchetti, L., Chevalier, A., Griffiths, R.-R., Salvatori, T., Lukasiewicz, T., Petersen, P. C., and Berner, J. (2023), "Mathematical Capabilities of Chatgpt." [10]

FUTU. (2024), "Futubull ai." [7]

GLM, T. (2024). "Chatglm: A Family of Large Language Models from glm-130b to glm-4 all tools." [4]

Google. (2025), "Data Science Agent in Colab with Gemini," available at *https://developers.googleblog.com/en/data-science-agent-in-colab-with-gemini/*. Accessed: 2025. [2,3,4]

Grosnit, A., Maraval, A., Doran, J., Paolo, G., Thomas, A., Beevi, R. S. H. N., Gonzalez, J., Khandelwal, K., Iacobacci, I., Benechehab, A., et al. (2024), "Large Language Models Orchestrating Structured Reasoning Achieve Kaggle Grandmaster Level," arXiv preprint arXiv:2411.03562. [4,5]

Guo, S., Deng, C., Wen, Y., Chen, H., Chang, Y., and Wang, J. (2024), "Ds-agent: Automated Data Science by Empowering Large Language Models with Case-based Reasoning," arXiv preprint arXiv:2402.17453. [4,5]

Hassan, M. M., Knipper, A., and Santu, S. K. K. (2023), "Chatgpt As Your Personal Data Scientist," arXiv preprint arXiv:2305.13657. [6]

Hong, S., Lin, Y., Liu, B., Wu, B., Li, D., Chen, J., Zhang, J., Wang, J., Zhang, L., Zhuge, M., et al. (2024), "Data Interpreter: An LLM Agent for Data Science," arXiv preprint arXiv:2402.18679 [4,5]

Hu, X., Zhao, Z., Wei, S., Chai, Z., Ma, Q., Wang, G., Wang, X., Su, J., Xu, J., Zhu, M., et al. (2024), "Infiagent-Dabench: Evaluating Agents on Data Analysis Tasks," arXiv preprint arXiv:2401.05507. [6]

Huang, Q., Vora, J., Liang, P., and Leskovec, J. (2024a), "Mlagentbench: Evaluating Language Agents on Machine Learning Experimentation." [6]

Huang, X., Liu, W., Chen, X., Wang, X., Wang, H., Lian, D., Wang, Y., Tang, R., and Chen, E. (2024b), "Understanding the Planning of LLM Agents: A Survey," arXiv preprint arXiv:2402.02716. [5]

IBM. (1968), "SPSS Statistics." [1]

Inala, J. P., Wang, C., Drucker, S., Ramos, G., Dibia, V., Riche, N., Brown, D., Marshall, D., and Gao, J. (2024), "Data Analysis in the Era of Generative AI," arXiv preprint arXiv:2409.18475. [1,10]

Inc., S. I. (1976), "SAS Software." [1]

Institute, M. G. (2011), *Big Data: The Next Frontier for Innovation, Competition, and Productivity*, San Francisco, CA: McKinsey & Company. [1]

Jiang, Z., Schmidt, D., Srikanth, D., Xu, D., Kaplan, I., Jacenko, D., Wu, Y., et al. (2024), "AIDE: The Machine Learning CodeGen Agent," available at *https://github.com/WecoAI/aideml*. Accessed: 2024-08-29. [4,5]

Jordan, M. I., and Mitchell, T. M. (2015), "Machine Learning: Trends, Perspectives, and Prospects," *Science*, 349, 255–260. [1]

Julius. (2025), "Julius ai." [7]

jupyterlab. (2023), "Jupyter-ai," available at *https://github.com/jupyterlab/jupyter-ai*. [3,4]

Kitchin, R. (2014), *The Data Revolution: Big Data, Open Data, Data Infrastructures and their Consequences*, Los Angeles, CA: Sage. [1]

Lai, Y., Li, C., Wang, Y., Zhang, T., Zhong, R., Zettlemoyer, L., tau Yih, S. W., Fried, D., Wang, S., and Yu, T. (2023), "Ds-1000: A Natural and Reliable Benchmark for Data Science Code Generation," in *Proceedings of the 40th International Conference on Machine Learning*, pp. 18319–18345. [6]

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. (2020), "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems* (Vol. 33), pp. 9459–9474. [6]

Li, Z., Zang, Q., Ma, D., Guo, J., Zheng, T., Niu, X., Yue, X., Wang, Y., Yang, J., Liu, J., et al. (2024), "Autokaggle: A Multi-Agent Framework for Autonomous Data Science Competitions," arXiv preprint arXiv:2410.20424. [4,5,6]

Liu, S.-C., Wang, S., Chang, T., Lin, W., Hsiung, C.-W., Hsieh, Y.-C., Cheng, Y.-P., Luo, S.-H., and Zhang, J. (2023), "Jarvix: A LLM No Code Platform for Tabular Data Analysis and Optimization," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 622–630. [6]

Luo, D., Feng, C., Nong, Y., and Shen, Y. (2024), "Autom3l: An Automated Multimodal Machine Learning Framework with Large Language Models," in *Proceedings of the 32nd ACM International Conference on Multimedia*, pp. 8586–8594. [4]

Ma, P., Ding, R., Wang, S., Han, S., and Zhang, D. (2023), "Insightpilot: An LLM-Empowered Automated Data Exploration System," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 346–352. [6]

MathWorks. (1984), "MATLAB." [1]

Microsoft. (1985), "Microsoft Excel." [1]

——— (2013), "Power BI." [1]

Nejjar, M., Zacharias, L., Stiehle, F., and Weber, I. (2023), "LLMs for Science: Usage for Code Generation and Data Analysis," *Journal of Software: Evolution and Process*, 37, e2723. [2]

OpenAI. (2023), "Gpt-4 Technical Report." arXiv preprint arXiv:2303.08774. [2,4]

——— (2024), "Gpt-4o System Card." [4,5]

Provost, F., and Fawcett, T. (2013), "Data Science and its Relationship to Big Data and Data-Driven Decision Making," *Big Data*, 1, 51–59. [1]

Qiao, B., Li, L., Zhang, X., He, S., Kang, Y., Zhang, C., Yang, F., Dong, H., Zhang, J., Wang, L., et al. (2023), "Taskweaver: A Code-First Agent Framework," arXiv preprint arXiv:2311.17541. [4,5]

Shen, Y., Song, K., Tan, X., Li, D., Lu, W., and Zhuang, Y. (2024), "Hugginggpt: Solving AI Tasks with Chatgpt and its Friends in Hugging Face," in *Advances in Neural Information Processing Systems* (Vol. 36). [7]

Steffensen, J. L., Dufault-Thompson, K., and Zhang, Y. (2016), "Psamm: A Portable System for the Analysis of Metabolic Models," *PLOS Computational Biology*, 12, 1–29. [2]

Sun, M., Han, R., Jiang, B., Qi, H., Sun, D., Yuan, Y., and Huang, J. (2024), "Lambda: A Large Model Based Data Agent," arXiv preprint arXiv:2407.17535. [2,4,5,6,12]

Trirat, P., Jeong, W., and Hwang, S. J. (2024), "Automl-Agent: A Multi-Agent LLM Framework for Full-Pipeline Automl," arXiv preprint arXiv:2410.02958. [4,5,6]

Tu, X., Zou, J., Su, W. J., and Zhang, L. (2023), "What Should Data Science Education Do with Large Language Models." arXiv preprint arXiv: 2307.02792. [2]

Waller, M. A., and Fawcett, S. E. (2016), "Data Science, Predictive Analytics, and Big Data: A Revolution that Will Transform Supply Chain Design and Management," *Journal of Business Logistics*, 37, 55–62. [1]

Wang, C., Lee, B., Drucker, S., Marshall, D., and Gao, J. (2024a), "Data Formulator 2: Iteratively Creating Rich Visualizations with AI," arXiv preprint arXiv:2408.16119. [4,6]

Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., and Ji, H. (2024b), "Executable Code Actions Elicit Better LLM Agents," arXiv preprint arXiv:2402.01030. [5,6]

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. (2022), "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems* (Vol. 35), pp. 24824–24837. [5]

Witten, I. H., Frank, E., and Hall, M. A. (2016), *Data Mining: Practical Machine Learning Tools and Techniques*, Amsterdam: Morgan Kaufmann. [1]

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., Li, B., Jiang, L., Zhang, X., and Wang, C. (2023), "Autogen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework," arXiv preprint arXiv:2308.08155. [4,6]

Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., et al. (2023), "The Rise and Potential of Large Language Model based Agents: A Survey," arXiv preprint arXiv:2309.07864. [6]

Xie, L., Zheng, C., Xia, H., Qu, H., and Zhu-Tian, C. (2024), "Waitgpt: Monitoring and Steering Conversational LLM Agent in Data Analysis with On-the-Fly Code Visualization," arXiv preprint arXiv:2408.01703. [4,6]

Xie, T., Zhou, F., Cheng, Z., Shi, P., Weng, L., Liu, Y., Hua, T. J., Zhao, J., Liu, Q., Liu, C., et al. (2023), "Openagents: An Open Platform for Language Agents in the Wild," arXiv preprint arXiv:2310.10634. [4,6]

Yang, Z., Zhou, Z., Wang, S., Cong, X., Han, X., Yan, Y., Liu, Z., Tan, Z., Liu, P., Yu, D., et al. (2024), "Matplotagent: Method and Evaluation for LLM-based Agentic Scientific Data Visualization," arXiv preprint arXiv:2402.11453. [6]

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. (2024), "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," in *Advances in Neural Information Processing Systems* (Vol. 36). [5]

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2022), "React: Synergizing Reasoning and Acting in Language Models," arXiv preprint arXiv:2210.03629. [5]

Zhang, C., Li, L., He, S., Zhang, X., Qiao, B., Qin, S., Ma, M., Kang, Y., Lin, Q., Rajmohan, S., et al. (2024), "UFO: A UI-focused Agent for Windows OS Interaction," arXiv preprint arXiv:2402.07939. [4]

Zhang, L., Zhang, Y., Ren, K., Li, D., and Yang, Y. (2023a), "Mlcopilot: Unleashing the Power of Large Language Models in Solving Machine Learning Tasks," arXiv preprint arXiv:2304.14979. [3,4,5,6]

Zhang, W., Shen, Y., Lu, W., and Zhuang, Y. (2023b), "Data-Copilot: Bridging Billions of Data and Humans with Autonomous Workflow," arXiv preprint arXiv:2306.07209. [4,5,6]

Zhou, X., Zhao, X., and Li, G. (2024), "LLM-Enhanced Data Management," arXiv preprint arXiv:2402.02643. [6]